KNN VS. K-MEANS

Eng. Maria Ayman

Learning Objectives

By the end of this session, students should be able to:

- Explain the fundamental concepts of K-Nearest Neighbors (KNN) and K-Means.
- Distinguish between supervised and unsupervised learning.
- Describe the main steps of both algorithms.
- Implement KNN and K-Means using Python and Scikit-learn.
- Evaluate model performance using appropriate metrics.
- Compare the algorithms and select an appropriate method for a given problem.

WHAT IS MACHINE LEARNING?

- Machine Learning (ML) is a branch of Artificial Intelligence that enables computers to learn patterns from data.
- ML algorithms use data to make predictions, identify relationships, and support decision-making.
- Instead of explicitly programming every rule, we train models to learn from examples.
- The performance of an ML model depends on the quality of the data, the selected algorithm, and the evaluation method.

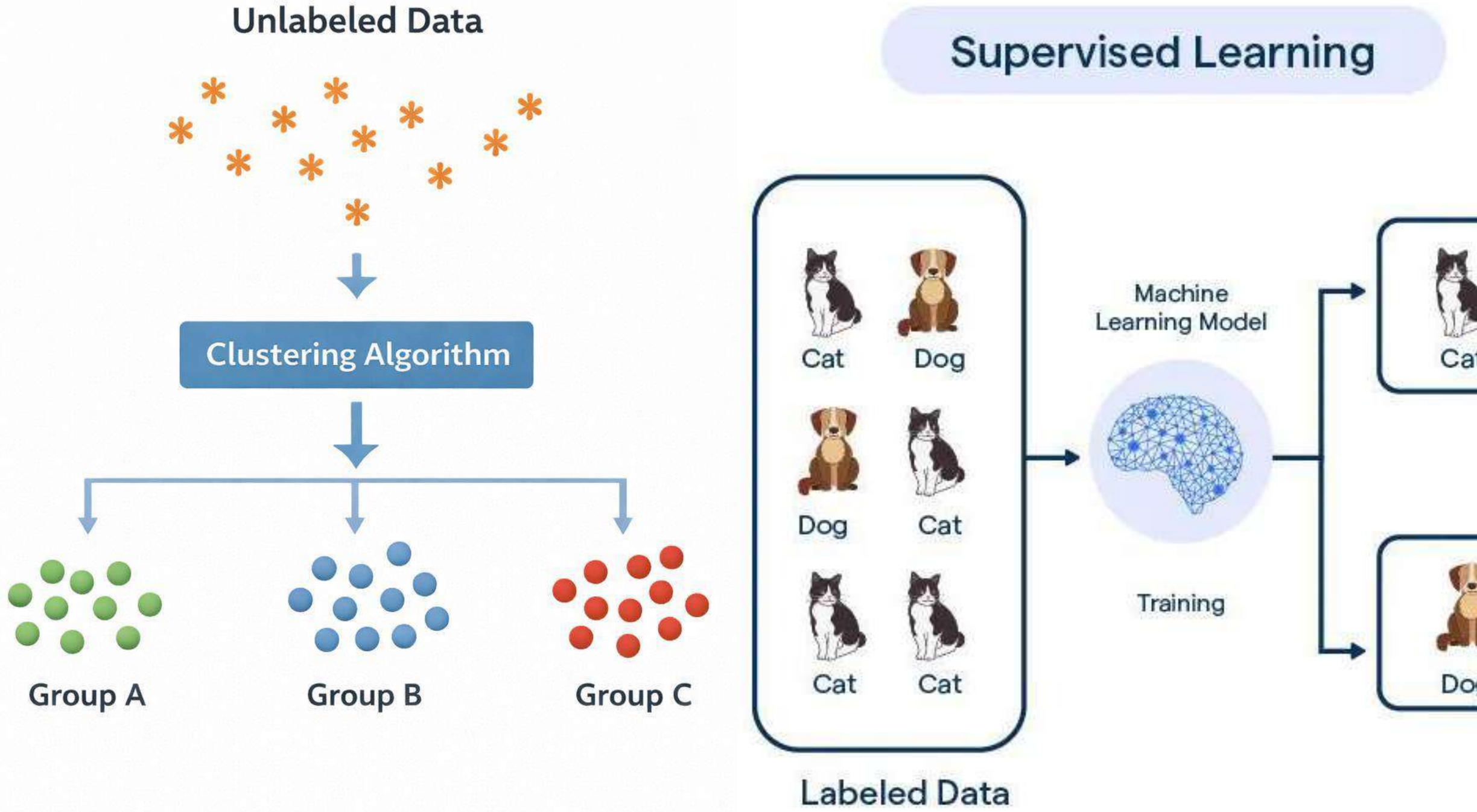
TYPES OF MACHINE LEARNING

Supervised Learning:

- Learns from labeled training data.
- Each training example contains input features and a corresponding target label or value.
- The objective is to predict the target for unseen data.
- Common tasks include classification and regression.
- Example: KNN

Unsupervised Learning:

- Learns from data without predefined target labels.
- Identifies hidden patterns, similarities, or structures in the data.
- Common tasks include clustering and dimensionality reduction.
- Example: K-Means


Unlabeled Data
Clustering Algorithm
Group A
Group B
Group C
Supervised Learning
Cat
Dog
Dog
Cat
Cat
Cat
Labeled Data
Machine Learning Model
Training
Cat
Dog

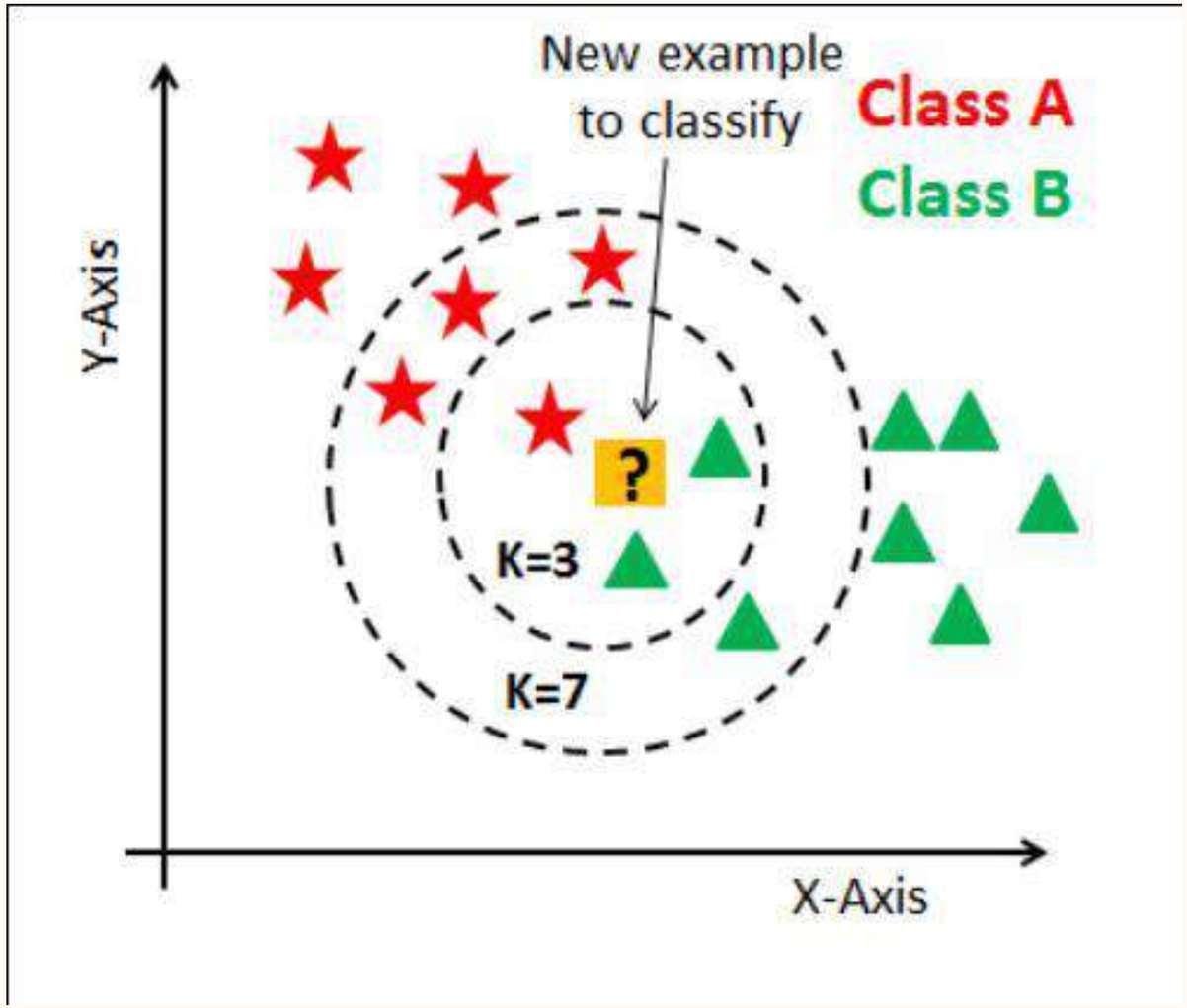
INTRODUCTION TO KNN

- K-Nearest Neighbors (KNN) is a supervised machine learning algorithm.
- It can be used for classification and regression tasks.
- KNN predicts the output of a new sample by examining its nearest training samples.
- The algorithm relies on similarity measures, commonly distance metrics.
- KNN is considered a lazy learning algorithm because it does not construct an explicit predictive model during training.

THE MAIN IDEA OF KNN

- Store the labeled training samples.
- Select the number of neighbors, represented by K.
- Calculate the distance between the new sample and the training samples.
- Identify the K nearest neighbors.
- For classification, assign the majority class among the selected neighbors (vote).
- For regression, typically calculate the mean of the neighbors' target values (average).

THE MAIN IDEA OF KNN



DISTANCE CALCULATION IN KNN

Euclidean Distance

- measures the straight-line distance between two points.
- It is commonly used to identify the nearest neighbors in numerical datasets.

For two points in a two-dimensional space:

$$d(A, B) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

- A smaller distance indicates greater proximity between two samples.
- Choice of distance metric may affect the algorithm's performance.

THE KNN ALGORITHM

1. Load the labeled dataset.
2. Split the dataset into training and testing sets.
3. Apply feature scaling when appropriate.
4. Select the value of K.
5. Calculate distances between the query sample and training samples.
6. Identify the K nearest neighbors.
7. Predict the class or target value.
8. Evaluate the predictions using suitable performance metrics.

SELECTING THE VALUE OF K

- K determines the number of neighbors considered during prediction.
- A small K makes the model more sensitive to noise and individual samples.
- A large K produces smoother decision boundaries but may overlook local patterns.
- Very small values of K may lead to overfitting.
- Very large values of K may lead to underfitting.
- The optimal value of K can be selected using validation data or cross-validation.
- An odd value of K may reduce ties in binary classification, but does not eliminate all possible ties.

FEATURE SCALING IN KNN

- KNN is a distance-based algorithm.
- Features with large numerical ranges may dominate distance calculations.
- Feature scaling places features on comparable scales.
- Standardization transforms features using their mean and standard deviation.
- The scaler must be fitted on the training data only.

$$z = \frac{x - \mu}{\sigma}$$

- The same fitted scaler must then be applied to the training and testing data.
- Proper scaling helps prevent data leakage and improves the reliability of evaluation.

ADVANTAGES AND LIMITATIONS OF KNN

Advantages

- Simple to understand and implement.
- Requires few assumptions about the underlying data distribution.
- Supports both classification and regression.
- Can model nonlinear decision boundaries.

Limitations

- Sensitive to feature scaling and irrelevant features.
- Sensitive to the choice of K and distance metric.
- Requires storing the training data for future predictions.

KNN IN PYTHON

- Library: Scikit-learn.
- Dataset: Iris Dataset.
- Model: KNeighborsClassifier.
- Main parameter: n_neighbors.
- Training method: fit(X_train, y_train).
- Prediction method: predict(X_test).
- Evaluation metrics: Accuracy, Precision, Recall, and F1-score.
- A confusion matrix can be used to analyze classification errors.

Load

Read features X and labels y

→

Split

Create train and test sets

→

Scale

Fit scaler on training data

→

Train

Fit KNN on scaled training data

→

Evaluate

Predict test labels and inspect metrics

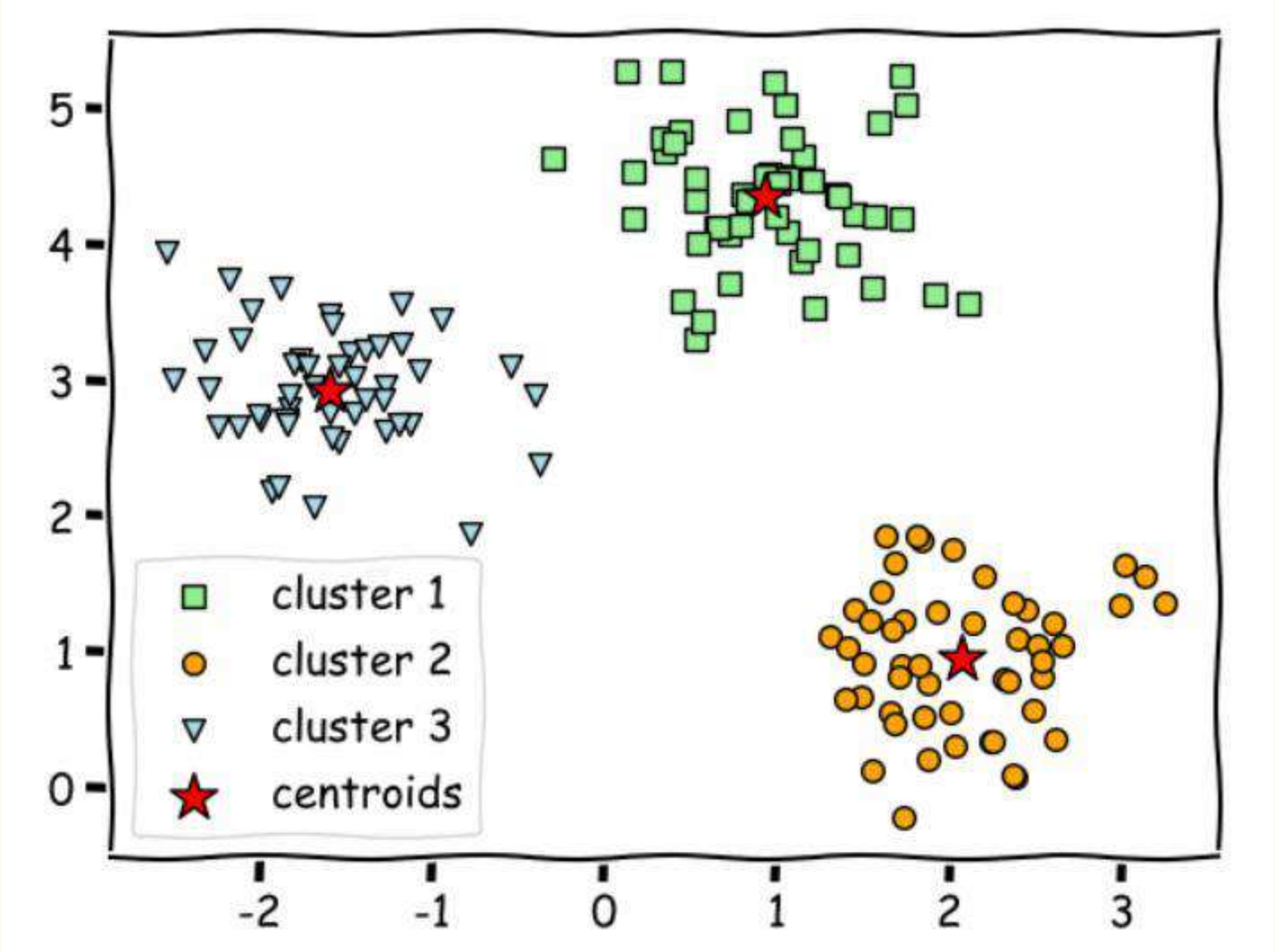
Never evaluate a model only on the same examples used to train it.

CODE TIME

INTRODUCTION TO K-MEANS

- K-Means is an unsupervised machine learning algorithm.
- It is primarily used for clustering numerical data.
- The algorithm partitions data into K clusters based on feature similarity.
- Each sample is assigned to the cluster with the nearest centroid.
- K-Means requires the number of clusters to be specified before fitting the model.
- The resulting cluster identifiers do not represent predefined class labels.

INTRODUCTION TO K-MEANS



THE MAIN IDEA OF K-MEANS

- Determine the required number of clusters, K.
- Initialize K cluster centroids.
- Assign each sample to its nearest centroid.
- Recalculate each centroid as the mean of the samples assigned to its cluster.
- Repeat the assignment and centroid-update steps.
- Stop when the algorithm converges or reaches its stopping criteria.

UNDERSTANDING CENTROIDS

- A centroid represents the mean position of the samples within a cluster.
- Each centroid contains one mean value for every feature.
- Centroids are recalculated after cluster assignments are updated.
- A centroid does not necessarily correspond to an actual observation in the dataset.
- The distance between a sample and a centroid helps determine cluster membership.

For a cluster containing n samples:

$$\mu_j = \frac{1}{n_j} \sum_{i=1}^{n_j} x_i$$

THE K-MEANS ALGORITHM

1. Select the number of clusters, K.
2. Initialize the cluster centroids.
3. Calculate the distance between each sample and the centroids.
4. Assign each sample to its nearest centroid.
5. Recalculate the centroids using the assigned samples.
6. Repeat the assignment and update steps.
7. Stop when the centroids or assignments stabilize, or a stopping criterion is reached.
8. Analyze the resulting clusters.

THE OBJECTIVE FUNCTION OF K-MEANS

- K-Means aims to minimize the within-cluster sum of squared distances.
- This objective is commonly referred to as Within-Cluster Sum of Squares (WCSS).
- In Scikit-learn, the corresponding quantity is available through the inertia_ attribute.
- Lower inertia indicates that samples are closer to their assigned centroids.
- Inertia generally decreases as the number of clusters increases.
- Therefore, inertia alone is insufficient to determine the optimal number of clusters.

$$J = \sum_{j=1}^{K} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

SELECTING THE NUMBER OF CLUSTERS

The Elbow Method

- Run K-Means using several values of K.
- Calculate the inertia for each value.
- Plot the number of clusters against the corresponding inertia.
- Identify the point where the rate of decrease in inertia becomes less pronounced.
- Select a reasonable K based on the trade-off between model simplicity and clustering quality.
- The elbow may not always be clearly identifiable.

HOW TO EVALUATE K-MEANS

- Inertia: Measures how close data points are to their cluster centers. Lower values indicate more compact clusters.
- Silhouette Score: Measures how well data points fit within their clusters compared with other clusters. Higher values generally indicate better-separated clusters.
- Choosing K: We can try different numbers of clusters and compare the results to select a suitable value.

Note: A lower Inertia does not always mean better clustering because it usually decreases when the number of clusters increases.

ADVANTAGES AND LIMITATIONS OF K-MEANS

Advantages

- Simple and computationally efficient for many numerical datasets.
- Easy to implement and interpret.
- Scales reasonably well to large datasets.
- Produces cluster centroids that summarize the groups.

Limitations

- Requires the number of clusters to be specified in advance.
- Sensitive to initialization and outliers.
- Sensitive to feature scaling.
- May converge to a local optimum.
- Works best for certain cluster shapes and distributions.
- May perform poorly when clusters have complex shapes, substantially different densities, or strongly different sizes.

K-MEANS IN PYTHON

- Library: Scikit-learn.
- Dataset: Iris Dataset.
- Model: KMeans.
- Main parameter: n_clusters.
- Training and clustering method: fit_predict(X).
- Cluster centers: cluster_centers_.
- Clustering objective: inertia_.
- Evaluation metrics: Silhouette Score and, when ground-truth labels are available, Adjusted Rand Index.
- The target labels must not be used as input when fitting the clustering model.

CODE TIME

KNN	K-Means
Supervised Learning	Unsupervised Learning
Uses labeled data	Uses unlabeled data during training
K = Number of neighbors	K = Number of clusters
Predicts a class or value	Groups similar data points
Accuracy	Inertia and Silhouette Score

PRACTICAL LAB TASKS

- Task 1: KNN
  - Run KNN with K = 3, 5, and 7.
  - Compare the Accuracy scores.
  - Identify the value of K with the best test performance.
- Task 2: K-Means
  - Run K-Means with K = 2, 3, and 4.
  - Compare the Silhouette Scores.
  - Observe how the number of clusters affects the results.
- Discussion
  - Why are the two algorithms different?
  - Which algorithm would you use when the data has no labels?

THANK YOU